Informe de Laboratorio 07

Tema: Autenticación, Sesiones y Seguridad en Express

Nota

Integrantes	Escuela	Asignatura
Michael Alexander Vasquez Villarreal Franklin Ronald Ulloa Chura Kevin Aldair Solis Florez Gabriel Alberto Coaguila Quicaña	Escuela Profesional de Ingeniería de Sistemas	Introducción al Desarrollo Web Semestre: II Código: 2501210

Laboratorio	Tema	Duración
07	Autenticación, Sesiones y Seguridad en Express	1 semana

Semestre académico	Fecha de inicio	Fecha de entrega
2026 - B	12 de octubre de 2026	XX de octubre de 2026

1. Tarea

Implementar en el avance del proyecto **UNSA Conecta** los siguientes objetivos:

- Implementar un flujo de autenticación seguro de usuarios (Registro, Login, Logout) en Express.js.
- Cifrar contraseñas mediante hashing unidireccional con salado utilizando la librería bcryptjs.
- Gestionar sesiones protegidas con express-session y configurar variables de entorno (dotenv) para la protección de credenciales.

2. Equipos, Materiales y Temas Utilizados

- Editor de código: Visual Studio Code.
- Entorno de ejecución: Node.js.
- Framework backend: Express.js.
- Librerías: `bcryptjs`, `express-session` y `dotenv`.
- Cliente API: Postman.
- Control de versiones: Git y GitHub.
- Frontend: HTML5, JavaScript nativo y Tailwind CSS.

3. URL de Repositorio GitHub

- **Repositorio completo:** https://github.com/MaicolVasquez/uc
- **Laboratorio 07:** https://github.com/MaicolVasquez/uc/blob/main/informes/IDWeb_lab07_avance_proyecto.pdf

4. Trabajo Colaborativo

El laboratorio se desarrolló dentro de un único repositorio de GitHub, utilizando ramas independientes para organizar los cambios y posteriormente integrar el trabajo en la rama principal. La distribución de roles en el proyecto es la siguiente:

Integrante	Rol en el proyecto
Michael Alexander Vasquez Villarreal	Desarrollador Backend
Franklin Ronald Ulloa Chura	Desarrollador Frontend
Kevin Aldair Solis Florez	Desarrollador Frontend
Gabriel Alberto Coaguila Quicaña	Desarrollador Backend

Tabla 1: Roles del equipo.

Código 1: Estructura de ramas utilizada en el trabajo grupal

```
main
|
|--- Garbriel
|
|--- Aldair
|
|--- Franklin_Ulloa
|
+--- feature/Michael_Vasquez
```

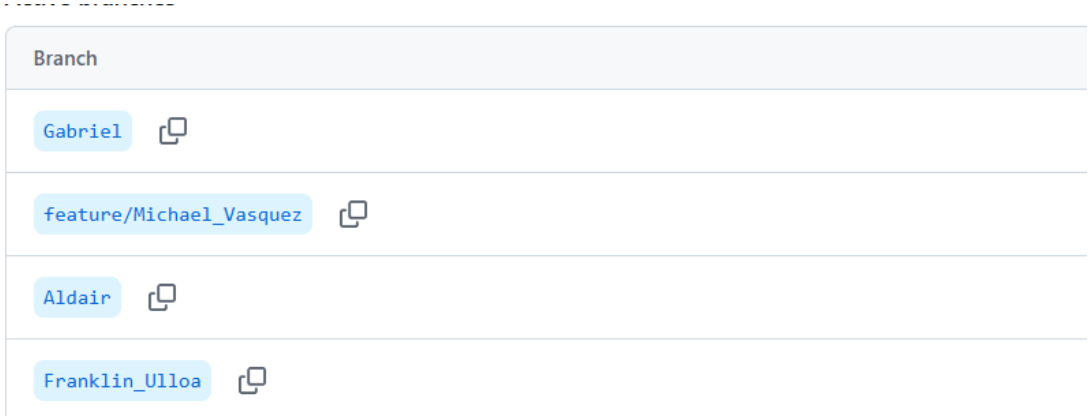


Figura 1: Historial de ramas y commits en GitHub.

5. Marco Conceptual

Autenticación y autorización. La autenticación verifica la identidad de un usuario (¿quién eres?), mientras que la autorización determina a qué recursos puede acceder (¿qué puedes hacer?). En este laboratorio el login resuelve la autenticación y el middleware `authMiddleware` resuelve la autorización sobre las rutas privadas.

Hashing con sal (bcrypt). Las contraseñas nunca se guardan en texto claro. Se transforman con una función hash unidireccional que incorpora una *sal* aleatoria y un *factor de costo* (salt rounds). El resultado es un digest irreversible, distinto incluso para contraseñas iguales y resistente a ataques de tablas arcoíris.

Sesiones y cookies. HTTP es un protocolo sin estado. Para recordar al usuario entre peticiones, el servidor guarda los datos de la sesión y entrega al navegador un identificador único dentro de una cookie firmada y marcada como `httpOnly`. En cada petición el navegador devuelve esa cookie y el servidor recupera la sesión.

Variables de entorno. Los secretos (por ejemplo, la clave que firma la cookie) no se escriben en el código fuente. Se guardan en un archivo `.env`, excluido de Git, y se leen con `process.env` mediante la librería `dotenv`.

6. Desarrollo del Laboratorio: Autenticación en UNSA Conecta

6.1. Estructura del proyecto

El módulo de autenticación se organizó separando configuración, rutas, middleware y persistencia de usuarios. El frontend se sirve desde el mismo servidor Express, de modo que la cookie de sesión funciona al ser del mismo origen.

Código 2: Estructura de directorios del proyecto

```
uc/
|--- backend/
|    |--- data/
|    |    |--- userStore.js      (almacen de usuarios)
|    |    +--- users.json        (generado en ejecucion, ignorado por Git)
|    |--- middleware/
|    |    +--- authMiddleware.js (proteccion de rutas)
|    |--- routes/
|    |    +--- auth.js           (register, login, perfil, logout)
|    |--- .env                   (secretos, ignorado por Git)
|    |--- .env.example           (plantilla sin secretos)
|    |--- .gitignore
|    |--- package.json
|    +--- server.js              (configuracion de Express y sesiones)
|--- fronend/
|    |--- cuentas.html          (formularios y perfil)
|    +--- pub/js/auth.js        (logica del cliente)
|--- bd/
+--- informes/
```

6.2. Gestión de variables de entorno

Se creó el archivo `.env` con el puerto y la clave de sesión. Para no exponer el secreto, el archivo se excluye del repositorio y se publica únicamente una plantilla (`.env.example`) sin valores reales.

Código 3: Archivo backend/.env.example (el .env real tiene el mismo formato)

```
PORT=3000
SESSION_SECRET=cambia_esto_por_una_clave_larga_y_aleatoria
```

Código 4: Archivo backend/.gitignore

```
node_modules/
.env
data/users.json
```

Explicación:

- `PORT=3000`: puerto en el que escucha el servidor.
- `SESSION_SECRET`: clave con la que `express-session` firma la cookie de sesión. Si alguien la conoce podría falsificar cookies, por eso debe ser larga, aleatoria y privada.
- `.env` dentro de `.gitignore`: evita que el secreto se suba a GitHub.

- `data/users.json` dentro de `.gitignore`: evita subir los hashes de los usuarios registrados durante las pruebas.
- `node_modules/`: las dependencias se reinstalan con `npm install`.

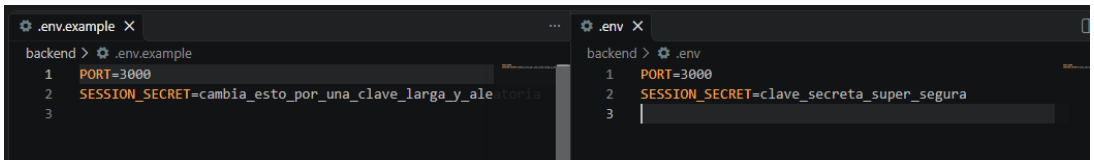


Figura 2: Archivos .env y .env.example en VS Code.

6.3. Configuración del servidor y de las sesiones (`server.js`)

Código 5: backend/server.js

```
const path = require('path');
require('dotenv').config({ path: path.join(__dirname, '.env') });

const express = require('express');
const session = require('express-session');
const authRoutes = require('./routes/auth');

const PORT = process.env.PORT || 3000;
if (!process.env.SESSION_SECRET) {
    console.error('Falta SESSION_SECRET en el archivo .env');
    process.exit(1);
}

const app = express();

app.use(express.json());
app.use(session({
    secret: process.env.SESSION_SECRET,
    resave: false,
    saveUninitialized: false,
    cookie: {
        httpOnly: true,
        sameSite: 'lax',
        secure: process.env.NODE_ENV === 'production',
        maxAge: 1000 * 60 * 60 * 2
    }
}));

// API
app.use('/api', authRoutes);

// Frontend servido por el mismo Express (mismo origen)
app.use(express.static(path.join(__dirname, '..', 'fronend')));

app.listen(PORT, () => {
    console.log(`UNSA Conecta en http://localhost:${PORT}`);
});
```

Explicación del código:

- `require('dotenv').config({ path: ... })`: carga las variables del archivo `.env` dentro de `process.env`. Se indica la ruta con `__dirname` para que funcione aunque el servidor se ejecute desde otra carpeta.
- `process.env.PORT || 3000`: usa el puerto del `.env` y, si no existe, el 3000.
- `if (!process.env.SESSION_SECRET) ... process.exit(1)`: el servidor se niega a arrancar sin secreto. Así nunca se usa una clave por defecto insegura.
- `express.json()`: middleware que convierte el cuerpo JSON de las peticiones en `req.body`.
- `session({...})`: registra el middleware de sesiones. Sus opciones:
  - `secret`: clave de firma de la cookie, tomada del entorno (no está escrita en el código).
  - `resave: false`: no vuelve a guardar la sesión si no cambió.
  - `saveUninitialized: false`: no crea sesión (ni cookie) para visitantes que no iniciaron sesión.
  - `httpOnly: true`: JavaScript del navegador no puede leer la cookie, lo que reduce el robo de sesión mediante XSS.
  - `sameSite: 'lax'`: el navegador no envía la cookie en peticiones entre sitios distintos (salvo navegación de nivel superior), lo que mitiga CSRF.
  - `secure`: es `true` solo en producción, para que la cookie viaje únicamente por HTTPS. En desarrollo local (HTTP) debe ser `false` o la cookie no se guardaría.
  - `maxAge`: duración de 2 horas (en milisegundos).
- `app.use('/api', authRoutes)`: monta las rutas de autenticación bajo el prefijo `/api`.
- `express.static(...)`: sirve las páginas de la carpeta `fronend` desde el mismo servidor.
- `app.listen`: inicia el servidor e imprime la URL en consola.

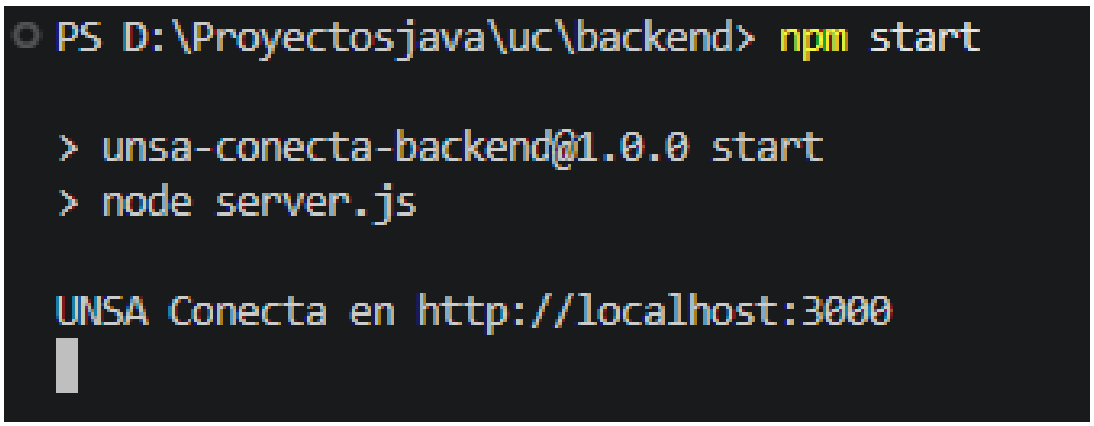


Figura 3: Servidor Express en ejecución.

6.4. Almacén de usuarios (`userStore.js`)

Siguiendo la guía, los usuarios se guardan en un arreglo en memoria. Como mejora, el arreglo se respalda en un archivo JSON para no perder las cuentas al reiniciar el servidor.

Código 6: backend/data/userStore.js

```
const fs = require('fs');
const path = require('path');

const FILE = path.join(__dirname, 'users.json');

function load() {
    try {
        const parsed = JSON.parse(fs.readFileSync(FILE, 'utf-8'));
        return Array.isArray(parsed) ? parsed : [];
    } catch (e) {
        return [];  // archivo inexistente o vacio
    }
}

const users = load(); // arreglo en memoria

function persist() {
    fs.writeFileSync(FILE, JSON.stringify(users, null, 2), 'utf-8');
}

module.exports = {
    findByEmail: (email) => users.find(u => u.email === email),
    findById: (id) => users.find(u => u.id === id),
    add(user) {
        users.push(user);
        persist();
        return user;
    }
};
```

Explicación del código:

- `load()`: lee `users.json` al iniciar. Si el archivo no existe o está vacío, devuelve un arreglo vacío gracias al `try/catch`.
- `const users = load()`: el arreglo en memoria con el que trabaja toda la aplicación.
- `persist()`: escribe el arreglo completo en disco cada vez que se agrega un usuario.
- `findByEmail` y `findById`: búsquedas que usan el login, el registro y el perfil.
- `add(user)`: agrega el usuario al arreglo y lo persiste.
- Al aislar estas funciones en un módulo, más adelante se podrá reemplazar el arreglo por una base de datos (Supabase) sin modificar las rutas.



Figura 4: Contraseña almacenada como hash en users.json.

6.5. Middleware de autorización (`authMiddleware.js`)

Código 7: backend/middleware/authMiddleware.js

```
// Protege rutas: si no hay sesion activa, corta el flujo con 401.
function authMiddleware(req, res, next) {
    if (!req.session || !req.session.userId) {
        return res.status(401).json({ error: 'Unauthorized: inicia sesión para continuar.'
            });
    }
    next();
}

module.exports = authMiddleware;
```

Explicación del código:

- `req.session.userId`: propiedad que solo existe si el usuario inició sesión correctamente.
- `!req.session || !req.session.userId`: si no hay sesión o no tiene `userId`, la petición no está autenticada.
- `return res.status(401).json(...)`: interrumpe el flujo y responde *401 Unauthorized*. El `return` es importante: evita que se ejecute `next()` después de responder.
- `next()`: si hay sesión válida, deja pasar la petición al siguiente manejador (el controlador de la ruta).

6.6. Registro de usuarios (`POST /api/auth/register`)

Código 8: backend/routes/auth.js - cabecera y registro

```
const express = require('express');
const bcrypt = require('bcryptjs');
const crypto = require('crypto');
const store = require('../data/userStore');
const authMiddleware = require('../middleware/authMiddleware');

const router = express.Router();
const SALT_ROUNDS = 10;
const EMAIL_RE = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
```

```
10
11  // Nunca devolver el hash al cliente
12  const publicUser = (u) => ({ id: u.id, email: u.email, nombre: u.nombre, createdAt:
        u.createdAt });
13
14  // POST /api/auth/register
15  router.post('/auth/register', async (req, res) => {
16      try {
17          const email = String(req.body.email || '').trim().toLowerCase();
18          const password = String(req.body.password || '');
19          const nombre = String(req.body.nombre || '').trim().slice(0, 60);
20
21          if (!EMAIL_RE.test(email)) return res.status(400).json({ error: 'Correo
                electrónico inválido.' });
22          if (password.length < 6) return res.status(400).json({ error: 'La contraseña debe
                tener al menos 6 caracteres.' });
23          if (store.findByEmail(email)) return res.status(409).json({ error: 'Ya existe una
                cuenta con ese correo.' });
24
25          const hash = await bcrypt.hash(password, SALT_ROUNDS);
26
27          // Re-chequeo: otro registro igual pudo completarse mientras se calculaba el hash
28          if (store.findByEmail(email)) return res.status(409).json({ error: 'Ya existe una
                cuenta con ese correo.' });
29
30          const user = store.add({
31              id: crypto.randomUUID(),
32              email,
33              nombre,
34              password: hash,
35              createdAt: new Date().toISOString()
36          });
37
38          res.status(201).json({ message: 'Cuenta creada correctamente.', user:
                publicUser(user) });
39      } catch (err) {
40          console.error(err);
41          res.status(500).json({ error: 'Error interno del servidor.' });
42      }
43  });
```

Explicación del código:

- `SALT_ROUNDS = 10`: factor de costo de bcrypt (10 rondas de salado, como pide la guía). Cada incremento duplica el tiempo de cálculo.
- `EMAIL_RE`: expresión regular que valida un formato básico de correo (texto, arroba, dominio y punto).
- `publicUser`: construye el objeto que se devuelve al cliente **sin** el campo `password`, para no exponer nunca el hash.
- `async (req, res) =>{ try ... catch }`: la ruta es asíncrona porque `bcrypt.hash` devuelve una promesa; el `try/catch` captura errores inesperados y responde 500.
- `String(req.body.email || '').trim().toLowerCase()`: convierte la entrada en texto, quita espacios y la pasa a minúsculas. Así `Ana@unsa.edu.pe` y `ana@unsa.edu.pe` son la misma cuenta, y se evitan errores si el campo llega vacío o con otro tipo.

- `.slice(0, 60)`: limita el nombre a 60 caracteres.
- Validaciones: correo inválido o contraseña menor de 6 caracteres devuelven *400 Bad Request*; correo repetido devuelve *409 Conflict*.
- `await bcrypt.hash(password, SALT_ROUNDS)`: genera una sal aleatoria y calcula el hash de forma asíncrona, sin bloquear el servidor. La sal y el costo quedan incluidos dentro del propio hash.
- Segundo `findByEmail`: como el hash tarda, otra petición con el mismo correo pudo registrarse mientras tanto; se vuelve a comprobar antes de guardar.
- `crypto.randomUUID()`: genera un identificador único no predecible para el usuario.
- `password: hash`: se guarda el **hash**, nunca la contraseña original.
- `res.status(201)`: código *201 Created*, con el usuario público en la respuesta.

6.7. Inicio de sesión (`POST /api/auth/login`)

Código 9: backend/routes/auth.js - login

```
// POST /api/auth/login
router.post('/auth/login', async (req, res) => {
    try {
        const email = String(req.body.email || '').trim().toLowerCase();
        const password = String(req.body.password || '');

        const user = store.findByEmail(email);
        const ok = user && await bcrypt.compare(password, user.password);

        if (!ok) return res.status(401).json({ error: 'Credenciales inválidas.' });

        // Nuevo ID de sesion al autenticar (previene session fixation)
        req.session.regenerate((err) => {
            if (err) return res.status(500).json({ error: 'No se pudo iniciar la sesión.'
                });
            req.session.userId = user.id;
            res.json({ message: 'Sesión iniciada.', user: publicUser(user) });
        });
    } catch (err) {
        console.error(err);
        res.status(500).json({ error: 'Error interno del servidor.' });
    }
});
```

Explicación del código:

- Se normaliza el correo igual que en el registro para encontrar la cuenta.
- `store.findByEmail(email)`: busca al usuario; si no existe, `user` es `undefined`.
- `await bcrypt.compare(password, user.password)`: compara de forma asíncrona la contraseña escrita con el hash guardado. bcrypt lee la sal y el costo del hash, vuelve a calcular y compara; nunca se "descifra" nada.
- `user && await ...`: si el usuario no existe, no se llama a `compare`.

- `if (!ok) ... 401`: tanto si el correo no existe como si la contraseña es incorrecta se devuelve el *mismo* mensaje genérico (`Credenciales inválidas.`). Así un atacante no puede averiguar qué correos están registrados (enumeración de usuarios).
- `req.session.regenerate(...)`: crea un nuevo identificador de sesión al autenticar, lo que previene el ataque de *session fixation* (que alguien fije el ID de sesión antes del login).
- `req.session.userId = user.id`: marca la sesión como autenticada, tal como exige la guía. `express-session` envía entonces la cookie `connect.sid` al navegador.
- La respuesta incluye `publicUser(user)`, de modo que el hash no se expone.

6.8. Recurso protegido (`GET /api/perfil`)

Código 10: backend/routes/auth.js - perfil protegido

```
// GET /api/perfil (protegido)
router.get('/perfil', authMiddleware, (req, res) => {
    const user = store.findById(req.session.userId);
    if (!user) {
        // La sesion apunta a un usuario que ya no existe
        return req.session.destroy(() => res.status(401).json({ error: 'Unauthorized:
            usuario no encontrado.' }));
    }
    res.json({ user: publicUser(user) });
});
```

Explicación del código:

- `router.get('/perfil', authMiddleware, ...)`: el middleware se ejecuta *antes* del controlador. Sin sesión, la petición se corta con 401 y nunca llega al código del perfil.
- `store.findById(req.session.userId)`: obtiene al usuario dueño de la sesión.
- `if (!user)`: caso borde, la sesión existe pero el usuario ya no está (por ejemplo, se borró `users.json`). Se destruye la sesión y se responde 401.
- `res.json({ user: publicUser(user) })`: devuelve los datos del usuario autenticado sin el hash.

6.9. Cierre de sesión (`POST /api/auth/logout`)

Código 11: backend/routes/auth.js - logout

```
// POST /api/auth/logout
router.post('/auth/logout', (req, res) => {
    req.session.destroy((err) => {
        if (err) return res.status(500).json({ error: 'No se pudo cerrar la sesión.' });
        res.clearCookie('connect.sid');
        res.json({ message: 'Sesión cerrada.' });
    });
});

module.exports = router;
```

Explicación del código:

- `req.session.destroy(...)`: elimina del servidor los datos de la sesión, por lo que el identificador de la cookie deja de ser válido.
- `res.clearCookie('connect.sid')`: indica al navegador que borre la cookie de sesión (`connect.sid` es el nombre por defecto de `express-session`).
- `module.exports = router`: exporta las rutas para montarlas en `server.js`.

6.10. Integración con el frontend (`cuentas.html` y `auth.js`)

La página `cuentas.html` contiene tres vistas: "verificando sesión", los formularios de login y registro, y la tarjeta de perfil. El script `auth.js` decide cuál mostrar consultando el endpoint protegido `/api/perfil`.

Código 12: fronend/pub/js/auth.js - cliente HTTP y arranque

```
const $ = (id) => document.getElementById(id);

// Cliente HTTP minimo: envia/recibe JSON y reenvia la cookie de sesion (mismo origen)
async function api(url, options = {}) {
    const res = await fetch(url, {
        credentials: 'same-origin',
        headers: { 'Content-Type': 'application/json' },
        ...options
    });
    const data = await res.json().catch(() => ({}));
    if (!res.ok) {
        const err = new Error(data.error || 'Ocurrió un error inesperado.');
        err.status = res.status;
        throw err;
    }
    return data;
}

// Al cargar: hay sesion activa?
document.addEventListener('DOMContentLoaded', async () => {
    showView('loading');
    try {
        const data = await api('/api/perfil'); // ruta protegida por authMiddleware
        renderProfile(data.user);
    } catch (err) {
        // 401 = sin sesion: mostrar formularios
        showView('forms');
        showAuthTab('login');
    }
});
```

Código 13: fronend/pub/js/auth.js - acciones de login, registro y logout

```
async function handleLogin(event) {
    event.preventDefault();
    clearError();
    try {
        const data = await api('/api/auth/login', {
            method: 'POST',
            body: JSON.stringify({
```

```
                email: $('loginEmail').value,
                password: $('loginPassword').value
            })
        });
        $('loginForm').reset();
        renderProfile(data.user);
    } catch (err) {
        showError(err.message);
    }
}

async function handleRegister(event) {
    event.preventDefault();
    clearError();

    const email = $('regEmail').value;
    const password = $('regPassword').value;

    if (password !== $('regPassword2').value) {
        return showError('Las contraseñas no coinciden.');
    }

    try {
        await api('/api/auth/register', {
            method: 'POST',
            body: JSON.stringify({ nombre: $('regName').value, email, password })
        });
        // Tras registrarse, inicia sesion automaticamente
        const data = await api('/api/auth/login', {
            method: 'POST',
            body: JSON.stringify({ email, password })
        });
        $('registerForm').reset();
        renderProfile(data.user);
    } catch (err) {
        showError(err.message);
    }
}

async function handleLogout() {
    try {
        await api('/api/auth/logout', { method: 'POST' });
    } finally {
        showView('forms');
        showAuthTab('login');
    }
}
```

Explicación del código:

- `api(url, options)`: función reutilizable que envía la petición con `fetch`, convierte la respuesta a JSON y lanza un error con el mensaje del servidor cuando el código HTTP no es 2xx.
- `credentials: 'same-origin'`: hace que el navegador envíe y guarde la cookie de sesión en las peticiones al mismo origen.
- `DOMContentLoaded`: al abrir la página se consulta `/api/perfil`. Si responde 200 se muestra el

perfil; si responde 401 (sin sesión) se muestran los formularios.

- `event.preventDefault()`: evita que el formulario recargue la página; el envío se hace con `fetch`.
- `handleRegister`: verifica en el cliente que las dos contraseñas coincidan, llama al registro y luego inicia sesión automáticamente con las mismas credenciales.
- `handleLogout`: llama a `POST /api/auth/logout` y vuelve a mostrar los formularios.
- Los mensajes de error y los datos del perfil se insertan con `textContent` (no con `innerHTML`), de modo que un nombre o mensaje con código HTML no se ejecuta y se evita XSS.

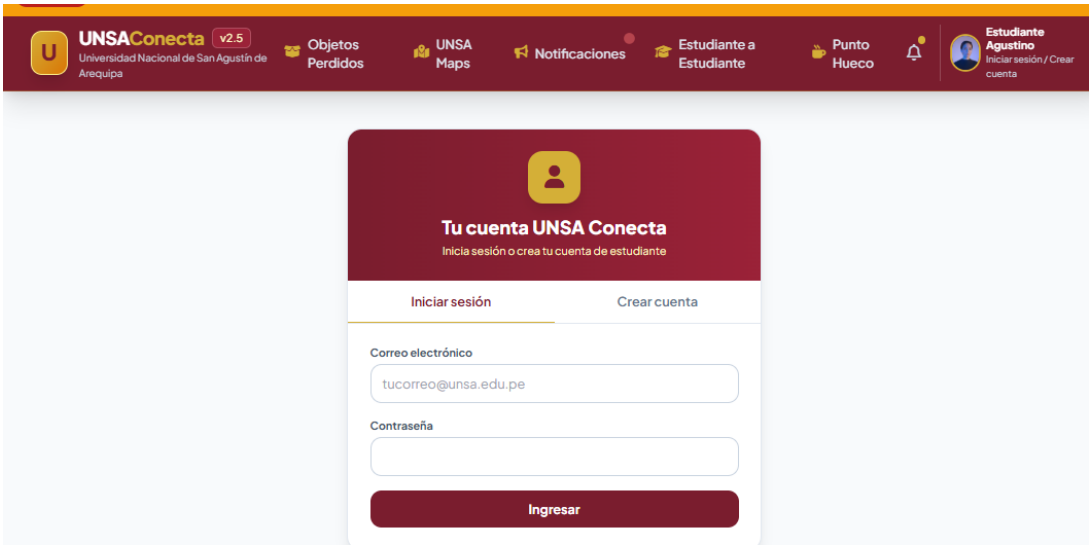


Figura 5: Página Mi cuenta: formularios de login y registro.

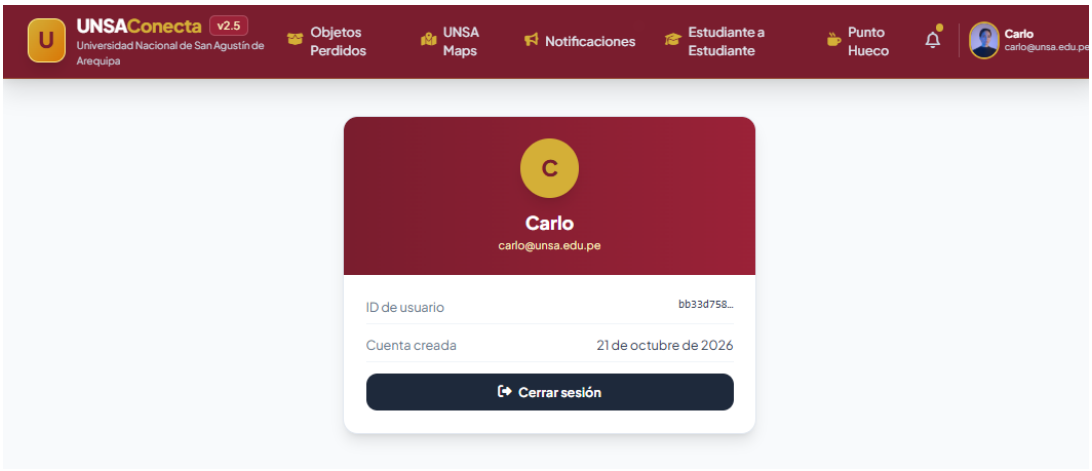


Figura 6: Página Mi cuenta: perfil del usuario autenticado.

7. Medidas de Seguridad Aplicadas

Medida	Cómo se implementó
Contraseñas con hash y sal	`bcrypt.hash` con 10 salt rounds; nunca se guarda ni se devuelve la contraseña original.
Comparación segura	`bcrypt.compare` asíncrono contra el hash almacenado.
Hash fuera de las respuestas	Función `publicUser` que solo expone id, email, nombre y fecha.
Secretos aislados	`SESSION_SECRET` y `PORT` en `.env`, ignorado por Git; plantilla `.env.example`.
Cookie protegida	`httpOnly`, `sameSite: 'lax'` y `secure` activado en producción.
Rutas privadas	`authMiddleware` corta con 401 si no existe `req.session.userId`.
Session fixation	`req.session.regenerate` al iniciar sesión.
Enumeración de usuarios	Mensaje único `Credenciales inválidas.` para correo o contraseña incorrectos.
Validación de entrada	Formato de correo, longitud mínima de contraseña, límite de longitud del nombre y correo normalizado.
XSS en el cliente	Uso de `textContent` en lugar de `innerHTML` al mostrar datos.

Tabla 2: Resumen de medidas de seguridad del módulo.

Limitaciones identificadas y mejoras futuras:

- El almacén de sesiones por defecto (`MemoryStore`) no es apto para producción: las sesiones se pierden al reiniciar y no escalan a varias instancias. Se recomienda un almacén externo como Redis o una tabla en la base de datos.
- Los usuarios aún no están en una base de datos; está previsto migrarlos a Supabase.
- No se implementó limitación de intentos de login (*rate limiting*) ni tokens CSRF; se recomienda añadirlos, junto con cabeceras de seguridad (por ejemplo, con `helmet`).

8. Cuestionario

1. **¿Por qué nunca deben almacenarse contraseñas en texto plano ni utilizar funciones de cifrado bidireccional (AES/RSA) en lugar de algoritmos de Hashing (bcrypt/Argon2)?**

   Texto plano. Si la base de datos se filtra (inyección SQL, copia de seguridad expuesta, empleado malintencionado), el atacante obtiene de inmediato todas las contraseñas. Como muchas personas reutilizan la misma contraseña en varios servicios, el daño se extiende más allá de la aplicación.

   Cifrado bidireccional. AES o RSA están diseñados para proteger la confidencialidad de datos que luego deben *recuperarse*, y por eso son reversibles con una clave. Para autenticar no hace falta recuperar la contraseña, solo comprobar que quien la escribe la conoce. Si se cifra, basta con que se filtre la clave (que normalmente vive en el mismo servidor) para descifrar todas las contraseñas a la vez.

   Hashing con sal. Una función hash es unidireccional: no existe una operación inversa, y la verificación consiste en volver a calcular el hash y compararlo. bcrypt añade además una sal

aleatoria única por contraseña (dos usuarios con la misma clave tienen hashes distintos, lo que inutiliza las tablas arcoíris) y un factor de costo configurable que hace lento cada intento, encareciendo la fuerza bruta. Argon2 va más allá al exigir también mucha memoria, lo que dificulta los ataques con GPU. Por el contrario, hashes rápidos de propósito general (como SHA-256 sin más) no son adecuados para contraseñas.

2. **¿Qué función cumplen los atributos de seguridad de una cookie: httpOnly, secure y sameSite en la prevención de ataques XSS y CSRF?**

   httpOnly. Impide que el código JavaScript de la página lea la cookie (`document.cookie`). No evita que ocurra un XSS, pero limita su impacto: un script inyectado no puede robar el identificador de sesión y enviarlo al atacante.

   secure. Indica al navegador que solo envíe la cookie por conexiones HTTPS. Así no viaja en claro por redes inseguras, lo que previene su captura mediante interceptación de tráfico (ataques de intermediario).

   sameSite. Controla si la cookie se adjunta a peticiones originadas desde otros sitios. Con `Strict` no se envía en ninguna petición entre sitios; con `Lax` solo en navegaciones de nivel superior con método seguro (como seguir un enlace); con `None` se envía siempre (y exige `secure`). Al no acompañar a las peticiones forjadas desde un sitio malicioso, mitiga los ataques CSRF, en los que se aprovecha la sesión abierta de la víctima para ejecutar acciones sin su consentimiento.

   Ninguna bandera sustituye a las demás: se complementan con otras defensas como escapar la salida HTML, una política CSP y tokens CSRF en operaciones sensibles.

3. **Explique la diferencia conceptual de escalabilidad entre la autenticación basada en Sesiones en Servidor (Stateful) y la autenticación basada en Tokens JSON Web Tokens - JWT (Stateless).**

   Sesiones en servidor (stateful). El servidor guarda el estado de cada sesión (en memoria, Redis o base de datos) y el cliente solo lleva un identificador opaco en una cookie. Ventajas: la sesión se puede invalidar al instante (logout, bloqueo de cuenta) y el cliente no ve datos internos. Costo de escalabilidad: cada petición requiere consultar ese estado, y con varias instancias del servidor hay que compartirlo (almacén común como Redis) o usar sesiones pegajosas (*sticky sessions*); además, el almacenamiento crece con el número de usuarios conectados.

   JWT (stateless). El token es un documento firmado que contiene las afirmaciones (*claims*) del usuario y su caducidad. El servidor solo verifica la firma y no necesita almacenar ni consultar nada, por lo que cualquier instancia puede validar cualquier token. Esto escala con facilidad de forma horizontal y encaja bien con microservicios y APIs consumidas por varios clientes. Costo: revocar un token antes de que expire es difícil (requiere listas negras, que reintroducen estado, o tokens de vida corta con *refresh tokens*); los datos del token pueden quedar desactualizados; y si el token se guarda en `localStorage` queda expuesto a robo por XSS, mientras que en una cookie `httpOnly` vuelve a requerir protección CSRF.

   En este proyecto se eligieron sesiones en servidor porque se trata de una aplicación web con un único backend y se prioriza poder cerrar sesiones de inmediato; al desplegarse con varias instancias habría que sustituir `MemoryStore` por un almacén compartido.

9. Conclusiones

- Se implementó un flujo completo de autenticación (registro, login, consulta de perfil y logout) en el proyecto UNSA Conecta usando Express.js, y se verificó con pruebas en Postman y desde la interfaz web.

- El uso de `bcryptjs` con sal y 10 rondas garantiza que las contraseñas se almacenen como hashes irreversibles, y la comparación asíncrona con `bcrypt.compare` permite validar el login sin conocer la contraseña original.
- `express-session` con cookie `httpOnly` y `sameSite`, junto con el middleware `authMiddleware`, permite proteger los recursos privados: sin sesión válida la API responde 401.
- Aislar la clave de sesión y el puerto en un archivo `.env` excluido del repositorio, y filtrar el hash en las respuestas con `publicUser`, evita la exposición de información sensible.
- Se identificó que, para producción, hace falta un almacén de sesiones externo, una base de datos real, limitación de intentos y protección CSRF; estos puntos quedan planteados como siguiente etapa del proyecto.

10. Rúbrica

10.1. Criterios de evaluación

Tabla 3: Rúbrica de evaluación del Laboratorio 07.

Criterios		Puntos	Check-list	Estudiantes	Profesor
Módulo de Autenticación y Hash (35 %)	Implementa la autenticación completa. Cifra contraseñas obligatoriamente con bcryptjs y compara hashes de forma asíncrona.	7	X	7	
Manejo de Sesiones y Protección (35 %)	Configura express-session con cookies httpOnly. Protege adecuadamente las rutas privadas mediante middlewares de autorización.	7	X	7	
Seguridad y Variables de Entorno (15 %)	Aísla completamente los secretos en .env. No expone datos sensibles ni contraseñas en las respuestas JSON devueltas.	3	X	3	
Cuestionario Teórico (15 %)	Demuestra conocimiento amplio en vectores de ataque OWASP, hashing, salado y banderas de protección de cookies.	3	X	3	
Total		**20**		**20**	

11. Referencias y Bibliografía Recomendada

- OWASP Foundation. (2025). *OWASP Top Ten Web Application Security Risks*. OWASP.org.
- OWASP Foundation. *Password Storage Cheat Sheet*. OWASP Cheat Sheet Series. OWASP.org.
- Node.js Security Working Group. (2024). *Security Best Practices for Express Applications*. Expressjs.com.
- Ristic, I. (2020). *Bulletproof SSL and TLS* (2nd ed.). Feisty Duck.
- Express.js Community. *express-session: Simple session middleware for Express*. Documentación oficial.
- *bcryptjs: Optimized bcrypt in JavaScript with zero dependencies*. Paquete npm.